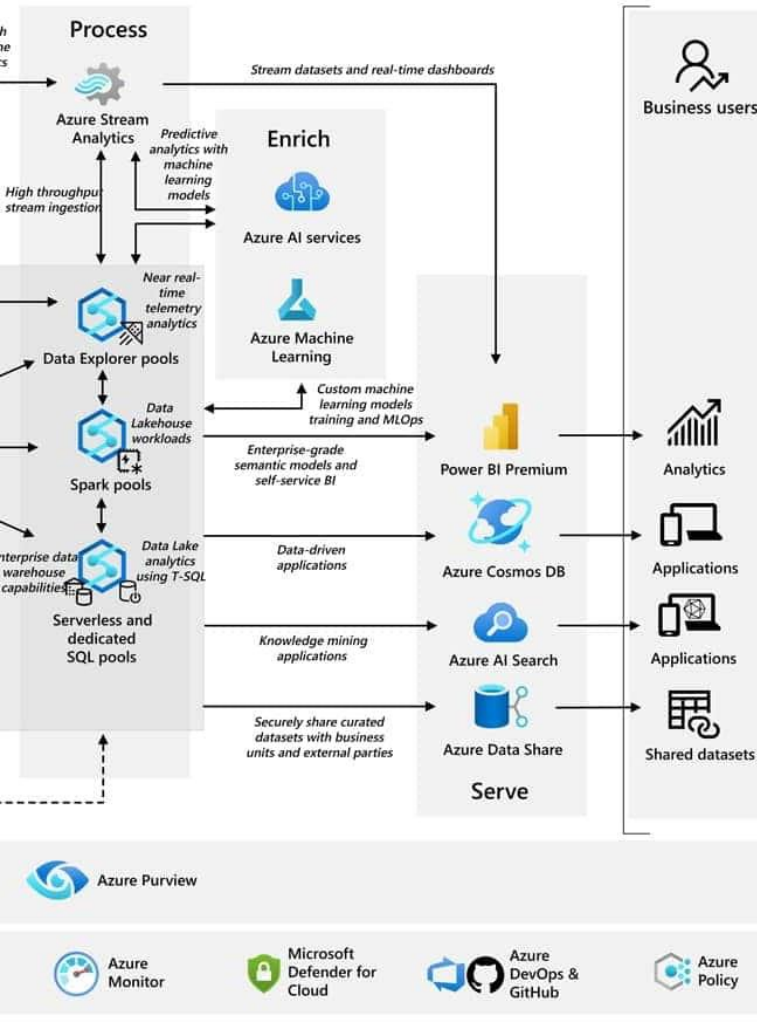




Diseño de la Arquitectura del Software

Semana 07

Ingeniería de Software 2

2026-1



Logro de la sesión



El estudiante será capaz de comprender y aplicar los conceptos fundamentales del ciclo de desarrollo de la arquitectura, identificando y especificando requerimientos arquitecturales y atributos de calidad, así como reconociendo la influencia de los drivers arquitectónicos y los estilos de arquitectura en el diseño de sistemas.

- ¿Alguna vez te has preguntado cómo se diseñan las aplicaciones que utilizas a diario para que sean rápidas y seguras?
- ¿Qué pasaría si tuvieras que construir un sistema desde cero?
- ¿Cómo influye la arquitectura de un software en su capacidad para adaptarse a cambios rápidos en un entorno ágil?



Contenidos

- Definición de Arquitectura de Software
- Ciclo de desarrollo de la Arquitectura de Software
- Drivers Arquitectónicos
- Escenario de calidad
- Estilos Arquitectónicos

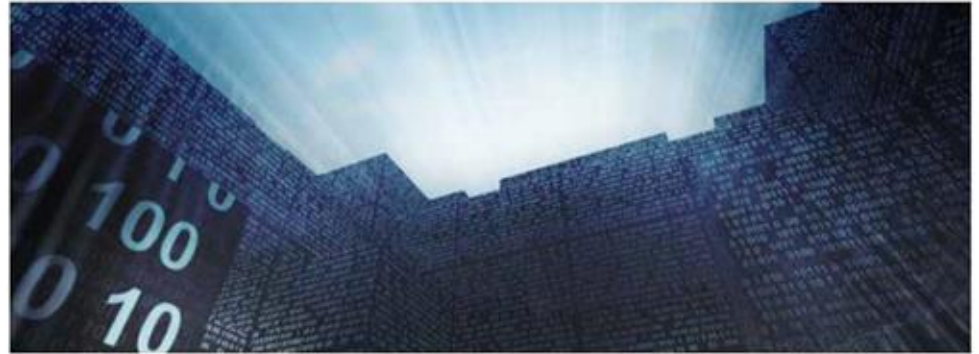
Proceso de diseño de la arquitectura

El resultado del proceso de diseño de la arquitectura es un modelo de arquitectura que describe cómo el sistema es organizado en un conjunto de componentes que están relacionados



Definiciones de Arquitectura de Software

Carnegie Mellon University
Software Engineering Institute



Según: Software Engineering Institute

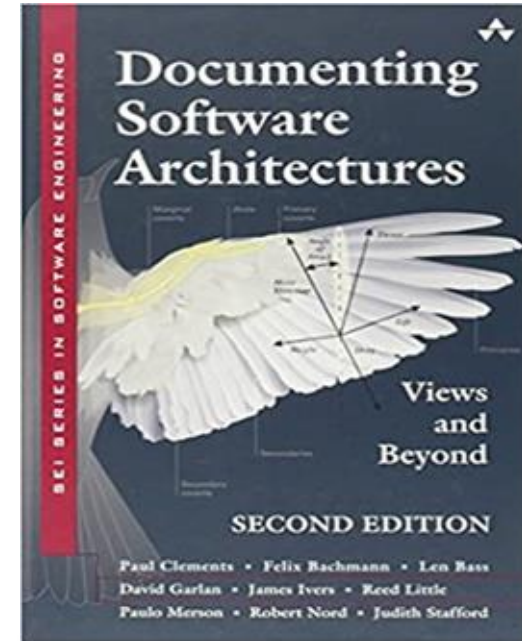
<https://resources.sei.cmu.edu/library/asset-view.cfm?assetID=513807>

What is your definition of software architecture?

Definiciones de Arquitectura de Software

Según el libro de “Documenting Software Architectures”

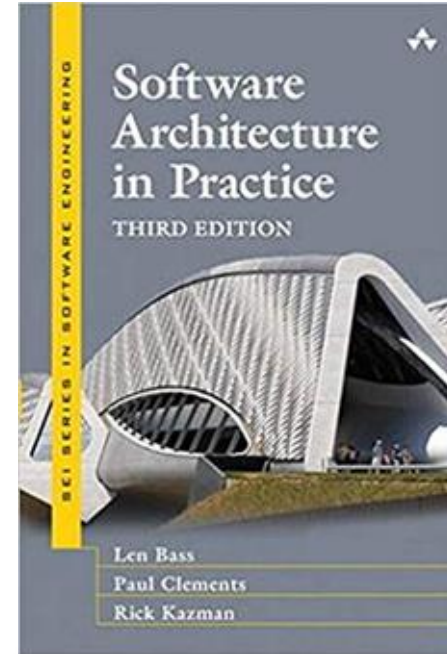
- El conjunto de estructuras necesarias para razonar sobre el sistema, que comprende elementos de software, relaciones entre ellos y propiedades de ambos.



Definiciones de Arquitectura de Software

Según el libro de "Software Architecture in Practice"

- La arquitectura de software de un programa o sistema informático es la estructura o estructuras del sistema, que comprenden elementos de software, las propiedades visibles externamente de esos elementos y las relaciones entre ellos.



Definiciones de Arquitectura de Software

Según el ANSI/IEEE Std1471-2000.

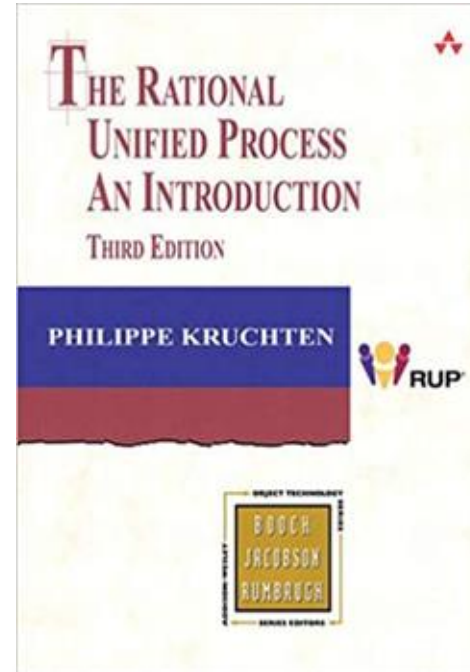
- La organización fundamental de un sistema, encarnada en sus componentes, sus relaciones entre sí y con el medio ambiente, y los principios que rigen su diseño y evolución.



Definiciones de Arquitectura de Software

Según Rational Unified Process, 1999 - Definición clásica

- Una arquitectura es el conjunto de decisiones significativas sobre la organización de un sistema de software, la selección de los elementos estructurales y sus interfaces por las que se compone el sistema, junto con su comportamiento según lo especificado en las colaboraciones entre esos elementos, la composición de estos elementos estructurales y de comportamiento en subsistemas progresivamente más grandes, y el estilo arquitectónico que guía esta organización---estos elementos y sus interfaces, sus colaboraciones, y su composición.



Definiciones de Arquitectura de Software

Según “Arquitectura según ISO/IEC/IEEE 42010:2011”

- La arquitectura de un sistema constituye lo esencial de ese sistema considerado en relación con su entorno. No existe una caracterización única de lo que es esencial o fundamental para un sistema; esa caracterización podría referirse a cualquiera o a la totalidad de:
 - componentes o elementos del sistema;
 - cómo se organizan o interrelacionan los elementos del sistema;
 - principios de la organización o diseño del sistema; y
 - principios que rigen la evolución del sistema a lo largo de su ciclo de vida.

ICS 35.080

ISO/IEC/IEEE 42010:2011

Systems and software engineering — Architecture description

THIS STANDARD WAS LAST REVIEWED AND
CONFIRMED IN 2017. THEREFORE THIS VERSION
REMAINS CURRENT.



Beneficios de la arquitectura

La arquitectura satisface los **atributos de calidad del sistema**. La arquitectura guía el proceso de implementación.

The Benefits of Software Architecting

*Peter Eeles
Executive IT Architect
IBM Rational Software*

In general terms, architecting is a key factor in reducing cost, improving quality, timely delivery against schedule, and delivery against requirements. In this article we focus on specific benefits that contribute to meeting these objectives.

It is often well known that, as an architect, you sometimes have to qualify an outcome. This article provides useful information for testing architecting as a critical part of the software development process.

Architecting addresses system qualities

The functionality of the system is supported by the architecture, through the interactions that occur between the various elements that comprise the architecture. However, one of the key characteristics of architecting is that it is the vehicle by which system qualities are achieved. Qualities such as performance, security, and maintainability cannot be achieved in the absence of a well-defined architectural vision since these qualities are not confined to a single architectural element, but rather pervade throughout the entire architecture.

For example, in order to address performance requirements, it may be necessary to consider the time for each component in the architecture to execute, and also the time spent in inter-component communication. Similarly, in order to address security requirements, it may be necessary to consider the nature of the communication between components, and consider specific "security control" components when necessary. All of these concerns are architectural and, in these examples, concern themselves with the individual components, and the connections between them.

A related benefit of architecting is that it is possible to answer such qualities early on in the project lifecycle. Architecture promotes an ability to answer in order to specifically ensure that such qualities are addressed. This is important since, as noted here, good architecture leads to good, executable software in the early test measures of whether the architecture has addressed such qualities.

Architecting drives consensus

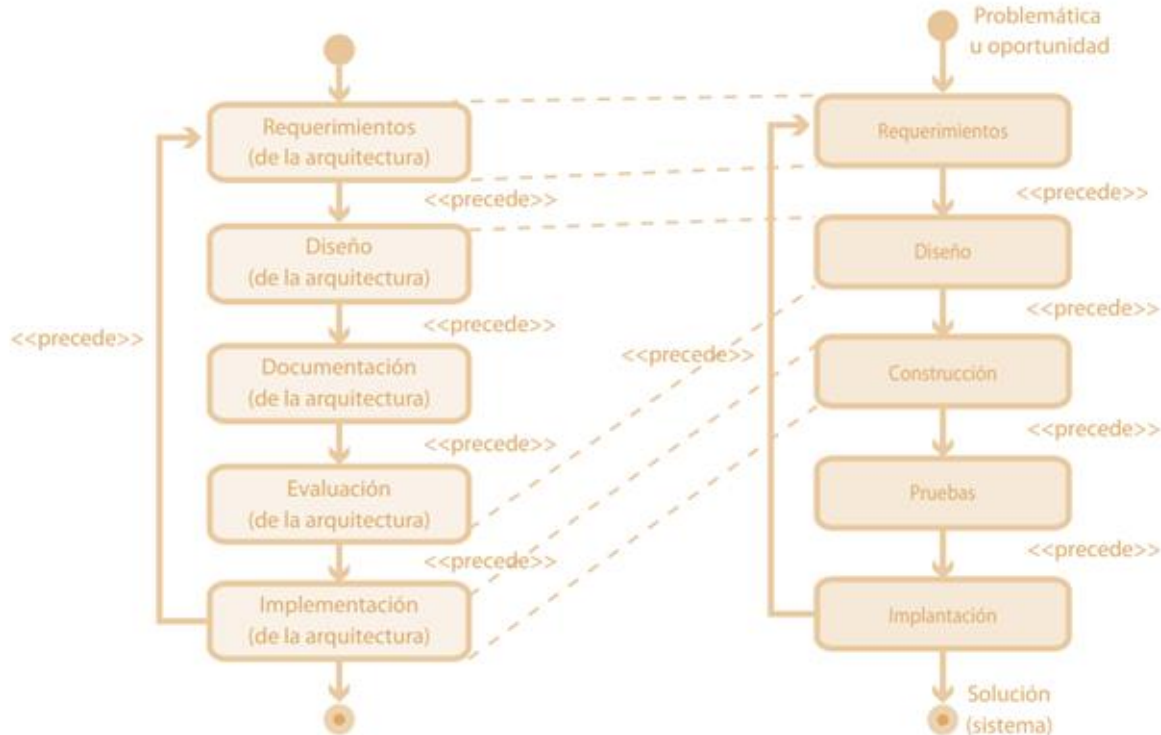
The process of architecting drives consensus between the various stakeholders, since it provides a vehicle for enabling debate about the system solution. In order to engage such debate, the process of architecting results in means that the architecture is clearly communicated and understood. An architecture that is effectively communicated allows discussion and visibility to be achieved, facilitates reviews, and allows agreement to be reached. Conversely, an architecture that is poorly communicated will not allow such debate to occur. Without such input, the resulting architecture is likely to be of lower quality.

Chapter 1, 1

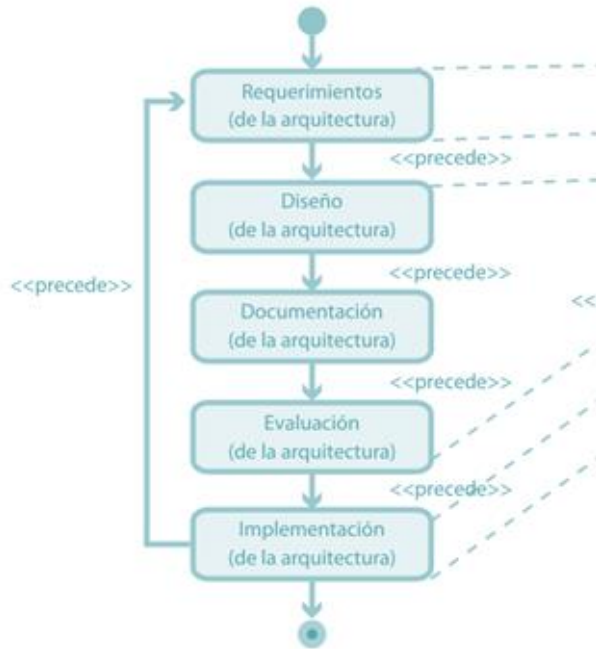
https://www.researchgate.net/profile/Peter-Eeles/2/publication/330202279_The_Benefits_of_Software_Architecting/links/5c3362eb92851c22a3624ca6/The-Benefits-of-Software-Architecting.pdf



Ciclo de desarrollo de la Arquitectura



Ciclo de desarrollo de la Arquitectura de Software



Método	Rol afectado	Etapa genérica del ciclo de la vida
QAW (Quality Attributes Workshops)	Analistas funcionales Arquitectos Stakeholders	Ing. de Requerimientos
ADD (Architectural Driver Design)	Arquitecto de Software	Análisis y diseño
ATAM (Architecture Trade Off Analysis Method)	Arquitectos Revisores técnicos	Análisis y diseño Verificación y Validación
Documentación mediante visas	Arquitectos	Análisis y diseño

El rol del Arquitecto de Software

developerWorks > Technical topics > Rational > Technical library >

Characteristics of a software architect

from The Rational Edge: If, in movie-making terms, the software project manager is the producer, since they make sure that things get done, then the software architect is the director who makes sure that things are done correctly and, ultimately, satisfy stakeholder needs. As the second of a four-part series, this article describes the role of software architect.

[► Comments](#)

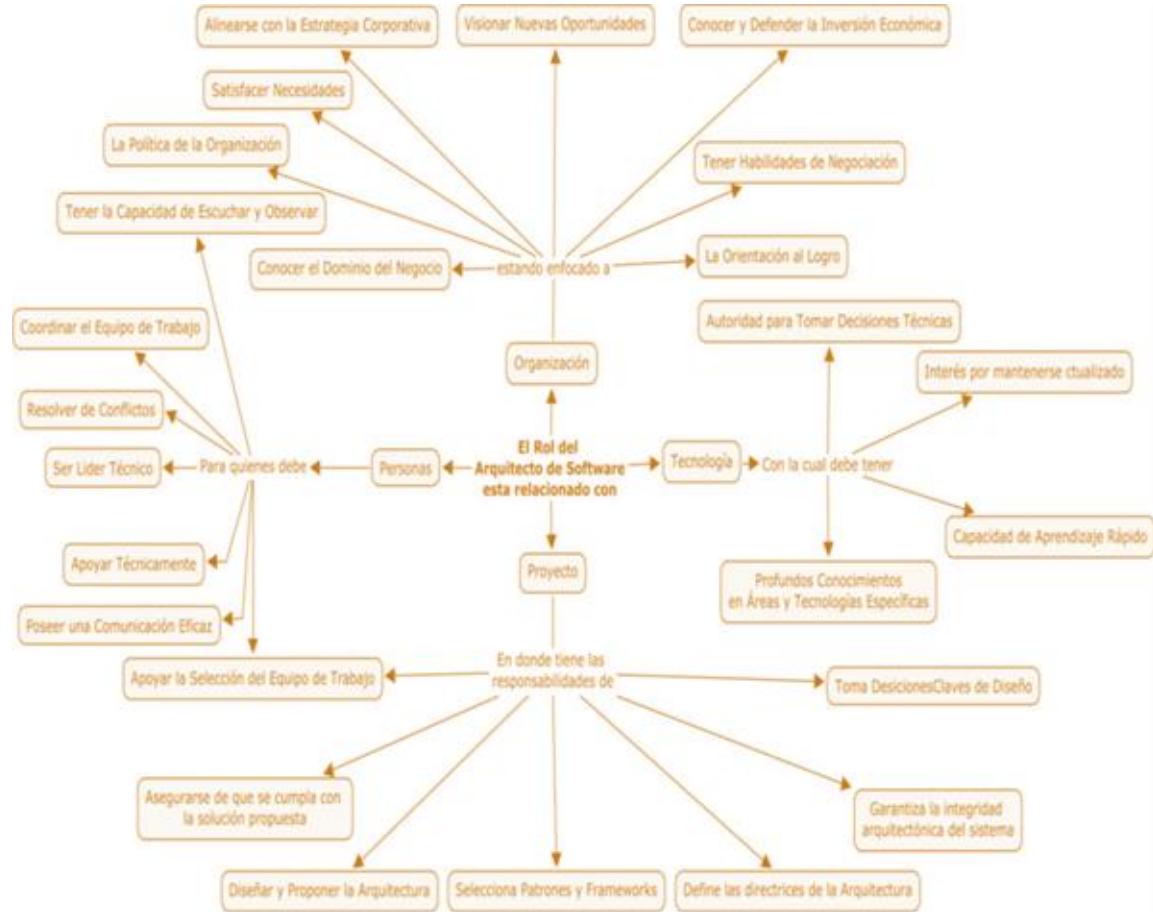
[Try related trial code](#)

This is the second article in a four-part series on software architecture. Last month, the [first article](#) defined what we mean by architecture. We can now turn our attention to the role that is responsible for the creation of the architecture -- the architect. The role of the architect is arguably the most

<https://research.cs.queensu.ca/home/emads/teaching/readings/CharacteristicsOfASoftwareArchitect.pdf>

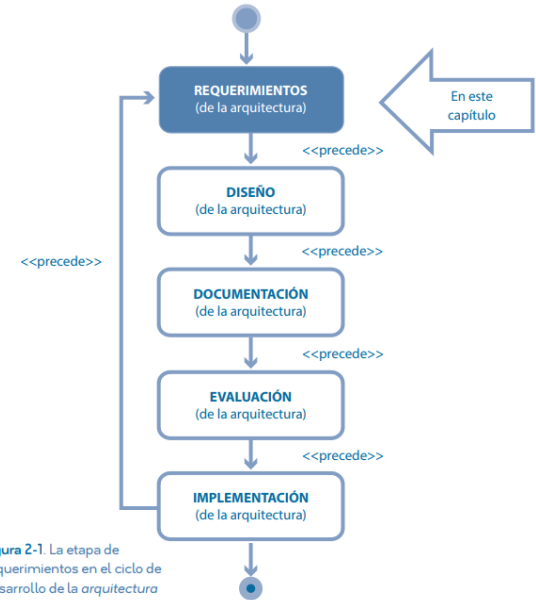
IEEE define el término "arquitecto":
Un arquitecto es la persona, equipo u organización responsable de la arquitectura de sistemas.

El Arquitecto de Software



1. La etapa de requerimientos en el ciclo de desarrollo de la ASw

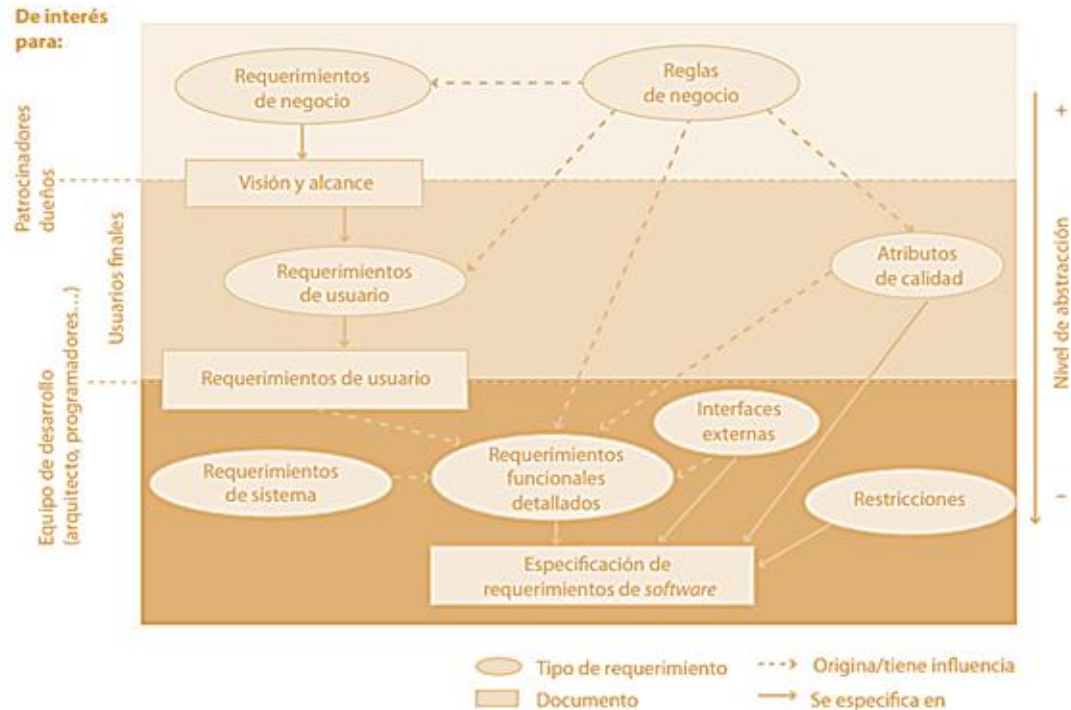
La identificación de los requerimientos ocurre durante el **análisis**. En el contexto de la ingeniería de software, la ingeniería de requerimientos es la disciplina que engloba las actividades relacionadas con la obtención, análisis, documentación y validación de estos.



› Figura 2-1. La etapa de requerimientos en el ciclo de desarrollo de la arquitectura de software

Requerimiento es una especificación que describe alguna funcionalidad, atributo o factor de calidad de un sistema de software. Puede describir también algún aspecto que restringe la forma en que se construye ese sistema.

2. Requerimientos : tipos y niveles de abstracción



© Humberto Cervantes Maceda / Pella Velasco Elzondo / Luis Castro Carreaga

Requerimientos arquitecturales

Requisito funcional:

Establecen lo que el sistema debe hacer y cómo debe comportarse o reaccionar ante los estímulos.

Todo requerimiento funcional se puede descomponer en entradas, lógica de procesamiento y salidas.

Se verifican a través de pruebas funcionales.

Requisito no funcional

Requerimientos que se expresan en términos de atributos de calidad (los terminados en -bility).

Requerimientos de usuario y requerimientos funcionales

Los requerimientos de usuario y los requerimientos funcionales : Los requerimientos de usuario especifican servicios que por lo habitual dan soporte a procesos de negocio que los usuarios podrán llevar a cabo mediante el sistema

- **Requerimientos de usuario primario :** describe servicios fundamentales que los usuarios desean llevar a cabo por medio del sistema,
- **Requerimientos de usuario :** describe acciones necesarias para dar soporte a la realización de los servicios especificados en los casos de uso o historias de usuario de los primarios.

Drivers arquitectónicos

Son el conjunto de los requerimientos de **mayor influencia** respecto de la forma que tomarán los elementos que componen las estructuras arquitectónicas. La etapa de requerimientos se centra en la identificación, documentación y priorización de drivers.

Estos drivers arquitectónicos se clasifican en tres clases:

1. Drivers funcionales.
2. Drivers de atributos de calidad.
3. Drivers de restricciones.

Identificación de drivers arquitectónicos

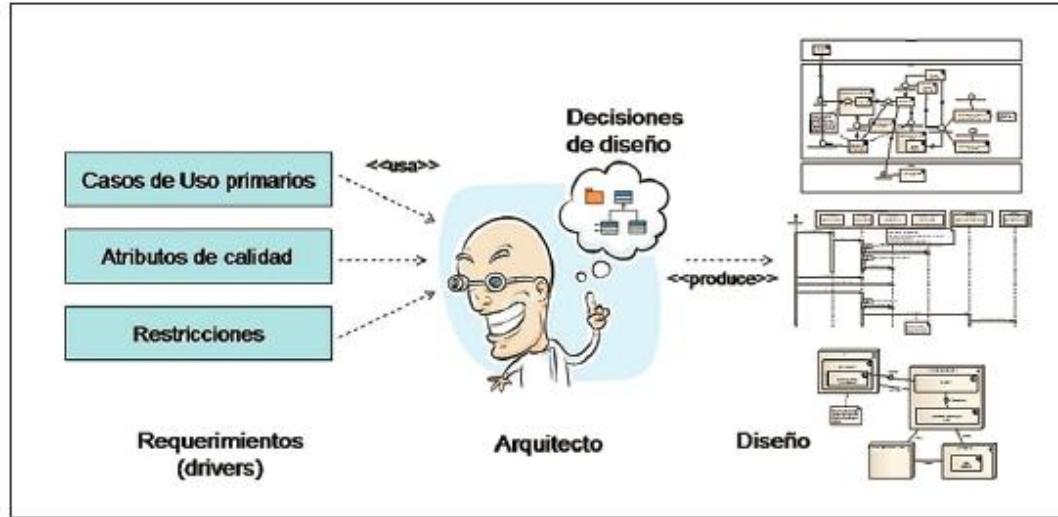


Figura 1: Durante el diseño de la arquitectura, el arquitecto toma como entrada los requerimientos que influyen la arquitectura (drivers) y produce un diseño arquitectónico.

Drivers arquitectónicos - Casos de uso primarios

Estos son los escenarios más importantes que el sistema debe manejar. Los casos de uso primarios ayudan a identificar las funcionalidades críticas que deben ser soportadas por la arquitectura, guiando así el diseño para asegurar que se satisfacen las necesidades del usuario.

Ejemplos :

- *Sistema de gestión de bibliotecas:* Un caso de uso primario podría ser "Registrar un nuevo libro", que implica funcionalidades como la entrada de datos del libro, la asignación de un identificador único y la actualización de la base de datos.
- *Aplicación de comercio electrónico:* Un caso de uso primario podría ser "Realizar una compra", que incluye la selección de productos, la gestión del carrito de compras y el procesamiento del pago.

Drivers arquitectónicos - Atributos de calidad

Estos son los requisitos no funcionales que afectan la calidad del sistema, como rendimiento, escalabilidad, seguridad, mantenibilidad y usabilidad. Los atributos de calidad son esenciales para evaluar cómo se comportará el sistema en diferentes condiciones y cómo se alineará con las expectativas de los stakeholders.

Ejemplos :

- **Rendimiento:** El sistema debe ser capaz de procesar 1000 transacciones por segundo sin degradar la experiencia del usuario.
- **Seguridad:** El sistema debe cumplir con estándares de seguridad como OWASP Top Ten, asegurando que no haya vulnerabilidades críticas.
- **Escalabilidad:** La arquitectura debe permitir la adición de nuevos servidores para manejar un aumento del 50% en la carga de usuarios sin necesidad de rediseñar el sistema.



Atributos de calidad

Los atributos de calidad especifican características útiles para establecer criterios sobre la calidad del sistema.



Drivers arquitectónicos - Restricciones

Las restricciones son limitaciones que deben ser consideradas durante el diseño arquitectónico. Estas pueden incluir restricciones técnicas (como el uso de ciertas tecnologías o plataformas), restricciones de tiempo y presupuesto, y regulaciones legales o normativas que el sistema debe cumplir.

Ejemplos

- **Restricciones técnicas:** El sistema debe ser desarrollado utilizando el framework .NET y debe ser compatible con bases de datos SQL Server.
- **Restricciones de tiempo:** El proyecto debe completarse en un plazo de seis meses debido a la fecha de lanzamiento del producto.
- **Regulaciones legales:** El sistema debe cumplir con la normativa GDPR para la protección de datos personales de los usuarios en la Unión Europea.



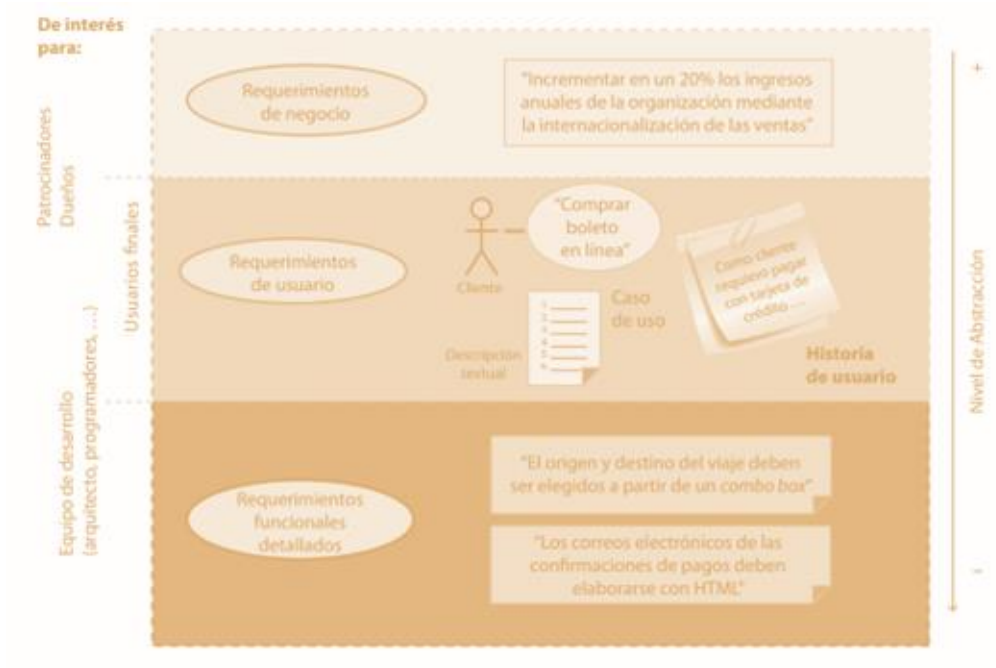
Drivers arquitectónicos - Restricciones

Las restricciones describen aspectos que limitan el proceso de desarrollo del sistema.

- **Restricciones técnicas** : se refieren a menudo a solicitudes expresas sobre el uso, durante el desarrollo del sistema, de productos de *software* provistos por terceros, métodos de diseño o implementación, productos de *hardware* o lenguajes de programación.
- **Restricciones administrativas**: describen aspectos que restringen el proceso de desarrollo del sistema. Estas restricciones a menudo se refieren a aspectos relacionados con el costo y tiempo de desarrollo, así como con el equipo de desarrollo.

Identificación de drivers arquitectónicos

Ejemplo de especificación de requerimiento



La extracción de drivers arquitectónicos

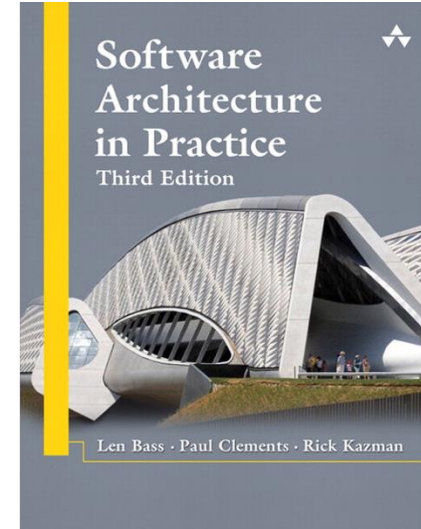
Documento
de visión y
alcance

Documento
de
requerimiento
s de usuario

Documento de
especificación
de
requerimientos

Requisito no funcional: Quality Attributes:

Un atributo de calidad en arquitectura de software se define como una **propiedad medible de un sistema** que indica qué tan bien este satisface las necesidades de las partes interesadas. Estos atributos, también conocidos como requisitos no funcionales, no afectan directamente la funcionalidad del sistema, pero determinan su calidad y características esenciales, como rendimiento, seguridad, mantenibilidad, entre otros.



Requisito no funcional: Escritura estilo tradicional

Atributo: Disponibilidad

“El sistema debe tener una **disponibilidad mensual** de 95 por ciento”

¿Es completo?

“La **disponibilidad** es la cantidad de tiempo que un sistema está funcionando conforme a lo previsto, durante período de tiempo predeterminado.”

 FailureHunter®
Menos Pérdidas. Mejor Desempeño.

Error Común al Calcular Disponibilidad

Descubra cuál es el error más común al calcular la Disponibilidad de equipos en su compañía.

Carlos J. Pernet
carlos.pernett@failurehunter.com
www.failurehunter.com



© FailureHunter, Inc. 2013

© FailureHunter 2013 | Disponibilidad - Error Común al Calcular Disponibilidad

<https://youtu.be/YQV70g9oBFA>

Requisito no funcional: Estilo de redacción moderno

Disponibilidad

Cuando en condiciones normales, se produzca una falla irrecuperable en el hardware, debe resultar posible restaurar el sistema a un estado conocido (no anterior a la copia de seguridad del día anterior) en menos de <xx> horas, desde el momento en que el hardware esté disponible.

El enfoque moderno permite encontrar rápidamente la mejor solución para el requisito. Existen catálogos de soluciones.

Escenario 3 - Desempeño

Descripción

Cuando un establecimiento comercial envía una publicación, en los momentos en los que hay mayor tráfico de anuncios (horas pico), las ofertas se hacen visibles para el usuario en la aplicación tras máximo diez (10) segundos de haber sido publicadas.

Validación del escenario

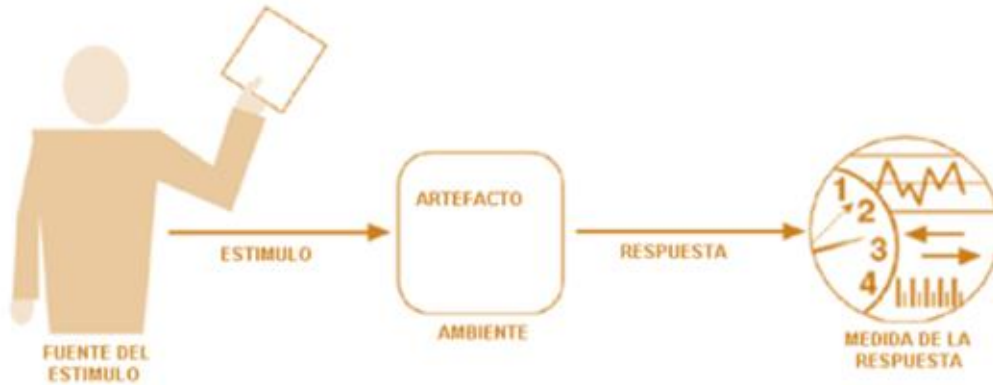
- **Origen del estímulo:** Establecimiento comercial.
- **Estímulo:** Enviar una publicación.
- **Entorno:** En los momentos que hay mayor tráfico de anuncios (horas pico).
- **Artefacto:** Las ofertas.
- **Respuesta:** Deben hacerse visibles para el usuario en la aplicación.
- **Medida de la respuesta:** Máximo diez (10) segundos después de haber sido publicadas.

<https://sites.google.com/site/wikinower/home/requerimientos-significativos-para-la-arquitectura>



Escenario de Calidad

Los escenarios de calidad son una forma de representar, de una manera formal, los QARs de un sistema. Un escenario consiste de seis parte.



Fuente: ¿quién?

Estímulo: ¿qué?

Artefacto: ¿dónde?

Medio ambiente: ¿cuándo?

Respuesta: ¿cuál?

Medir: ¿cómo?

Escenarios de calidad

Son Requisitos de Calidad

Fuente del estímulo (Source):	Quién o qué genera el evento? (Un usuario interno, un hacker, un pico de tráfico, un servidor que se cae)
Estímulo (Stimulus)	¿Qué es lo que ocurre? (Se intenta iniciar sesión, se cae la base de datos, llegan 5,000 peticiones en un segundo)
Artefacto (Artifact)	¿Qué parte del sistema recibe el impacto? (El proceso de autenticación, el servidor web, la base de datos principal)
Entorno (Environment)	¿En qué condiciones ocurre esto? (En operación normal, durante el mantenimiento, en un momento de sobrecarga)
Respuesta (Response)	¿Qué hace el sistema como reacción? (Bloquea la cuenta, cambia al servidor de respaldo, procesa las peticiones)
Medida de la respuesta (Response Measure)	¿Cómo medimos el éxito de esa respuesta? Debe ser cuantitativo (en menos de 2 segundos, con 0% de pérdida de datos, 99.9% de uptime)

Ejemplo

Rendimiento: Si hay 20 clientes simultáneamente, el tiempo de respuesta debería ser menos que 1 segundo en circunstancias normales



Ejemplo

Los usuarios inician 1.000 transacciones X por minuto (probabilidad) bajo condiciones normales, de 9 a 18:00hs, el sistema debe procesarlas (resultado en pantalla) en una latencia menor a 3 segundos.

Elemento	Descripción
Origen del Estímulo.	Usuarios. Definir el tipo de Usuario si es necesario.
Estímulo.	Inicio de Tx X. Probabilístico. 1.000 tx por minuto
Ambiente.	Procesamiento Normal. De 9 a 18. Carga Normal.
Componentes.	Todo el Sistema
Respuesta.	Transacción procesada
Medida de la Respuesta.	la latencia debe ser menor a 3 segundos

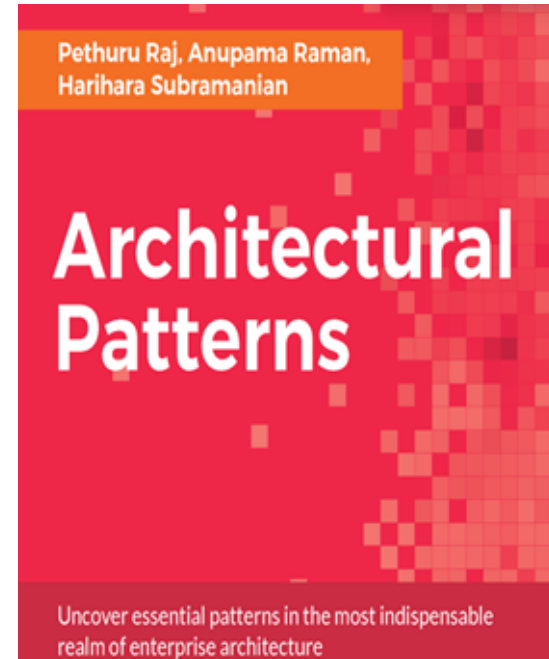
Estilos de arquitectura

Es una especialización de elementos y tipos de relaciones, junto con un conjunto de restricciones sobre cómo se pueden usar. Proporciona la organización de alto nivel del software



Patrones arquitectónicos de Software

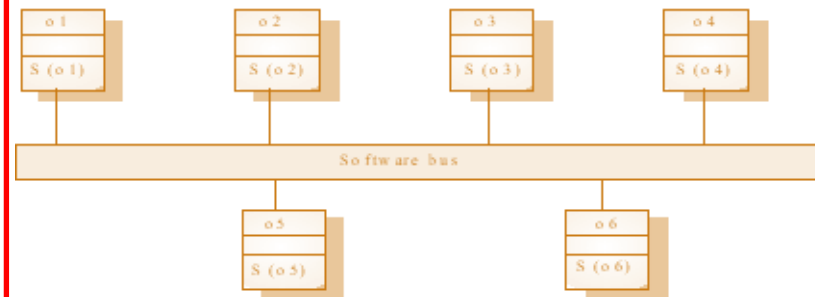
los patrones arquitectónicos se utilizan para decidir hábilmente la arquitectura estable de un sistema en desarrollo. Los patrones arquitectónicos permiten tomar una serie de decisiones en cuanto a la elección de tecnologías y herramientas.



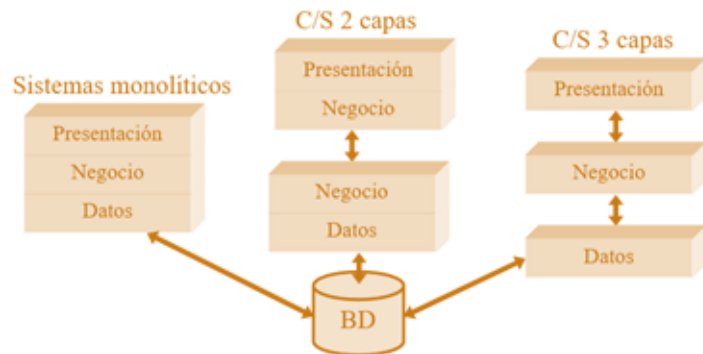
Estilos arquitectónicos



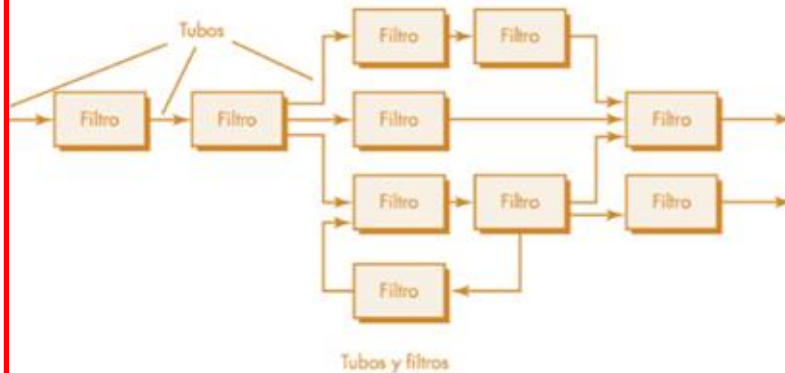
El modelo de repositorio



Objetos Distribuidos

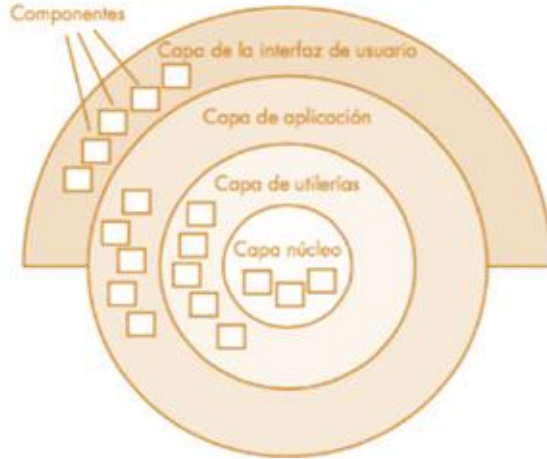


Cliente-servidor

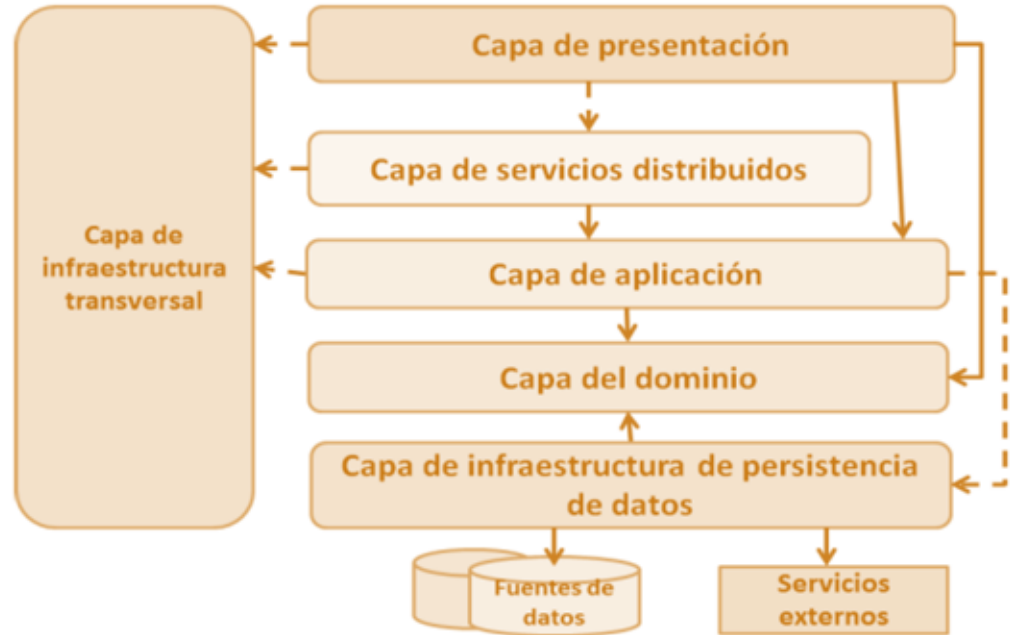


Tuberías y filtros

Estilos arquitectónicos

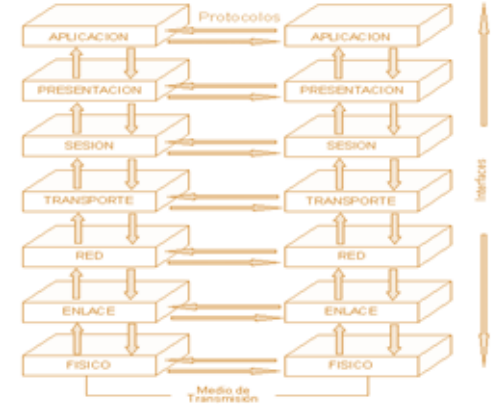
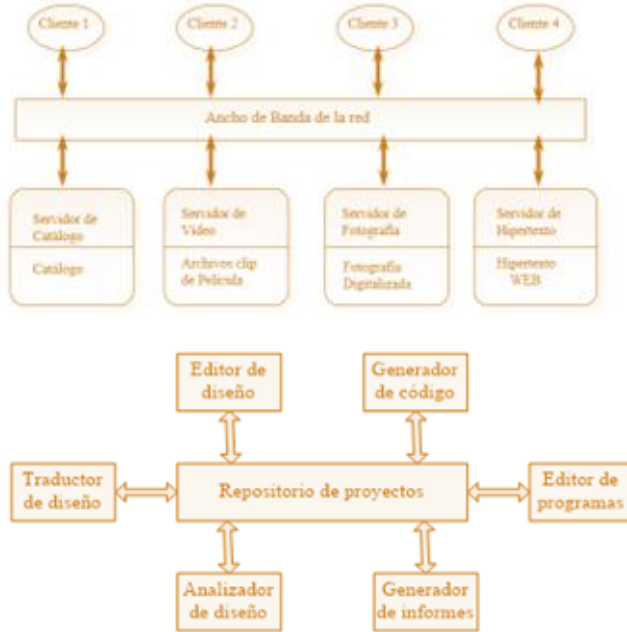


Arquitectura n de Capas



Arquitectura N-Capas
orientada al Dominio

Ejemplos de Aplicación



Object-oriented architecture (OOA)

- OOA: Herencia, polimorfismo, encapsulación y composición
- Producción de aplicaciones de software altamente modulares (altamente cohesivas y poco acopladas), utilizables y reutilizables.

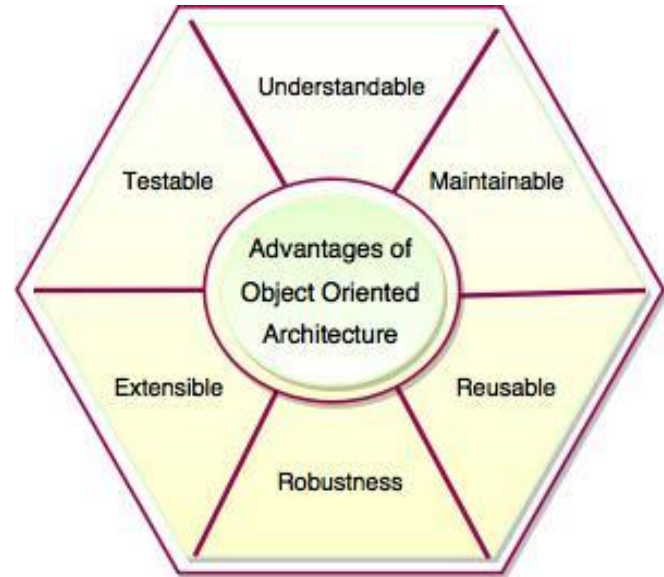
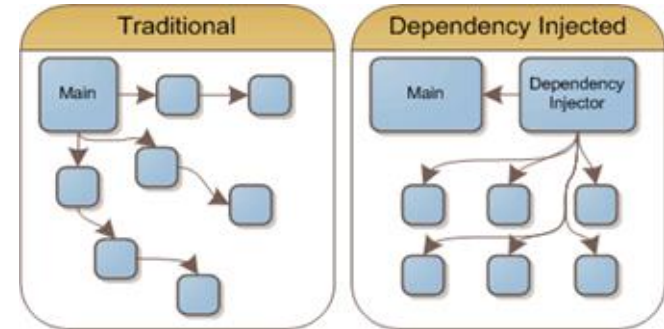
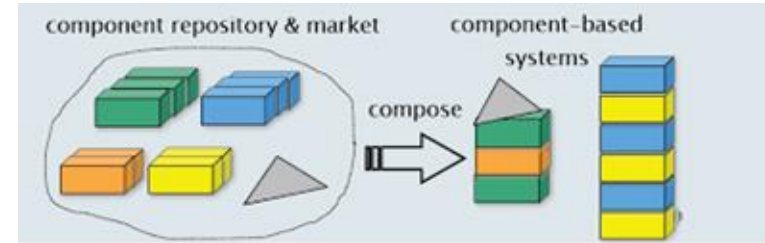


Fig. Advantages of Object Oriented Architecture

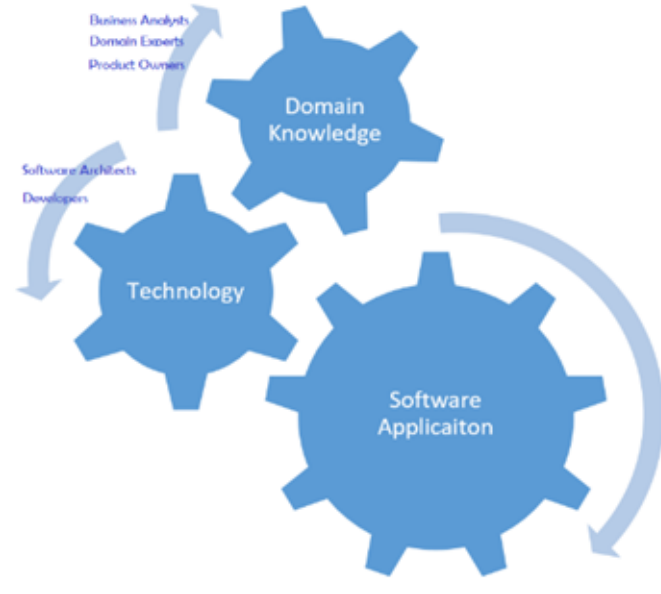
Component-based assembly (CBD) architecture

- Bloque de creación para diseñar y desarrollar aplicaciones a escala empresarial
- Los componentes exponen interfaces bien definidas
- Los componentes son reutilizables, reemplazables, sustituibles, extensibles, independientes
- **La inyección de dependencias** (DI) administra las dependencias entre componentes



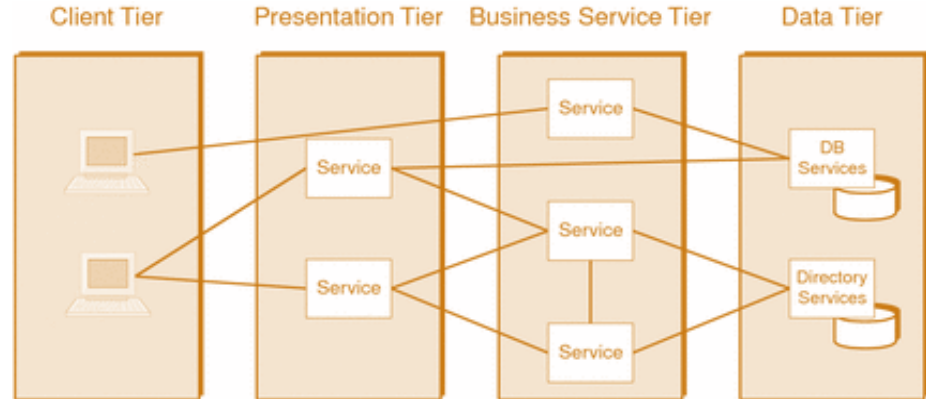
Domain-driven design (DDD) architecture

- es un enfoque orientado a objetos para diseñar software basado en el dominio comercial
- El modelo de dominio se puede ver como un Framework del cual se pueden preparar y racionalizar las soluciones.



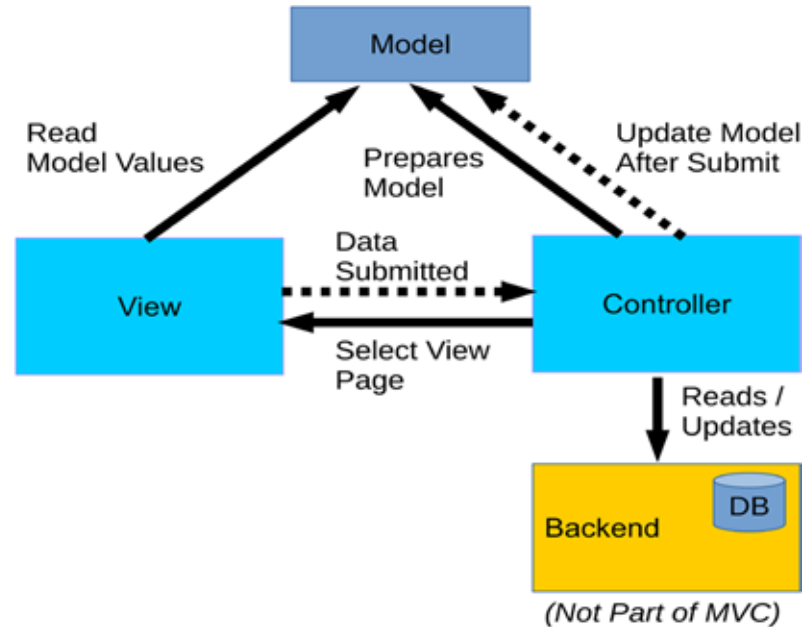
Multi-tier distributed computing architecture

- Los componentes de la aplicación se pueden implementar en varias máquinas.
- Alta disponibilidad, escalabilidad, manejabilidad
- Las aplicaciones web, en la nube y móviles se implementan utilizando esta arquitectura.



MVC Pattern

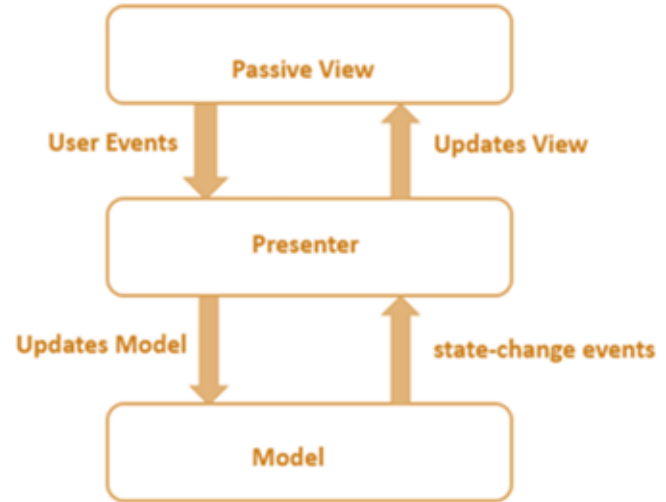
- **Modelo:** Gestiona los datos de una aplicación.
- **Vista:** describe la presentación de los datos y los elementos de control al usuario.
- **Controlador:** maneja la entrada del usuario y prepara el conjunto de datos necesario para que la parte de la vista haga su trabajo



The model view presenter (MVP) pattern

es un estilo de arquitectura de software que se utiliza principalmente en el desarrollo de aplicaciones de interfaz de usuario. Este patrón se basa en la separación de preocupaciones, dividiendo la aplicación en tres componentes principales:

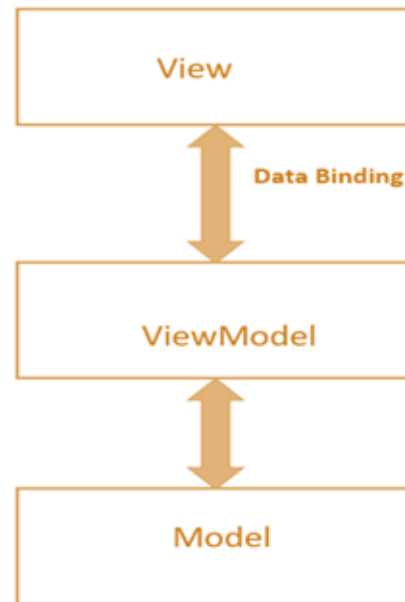
- **Modelo** : Gestiona la lógica de negocio y los datos de la aplicación, realizando operaciones y notificando cambios a la vista.
- **Vista** :Presenta la interfaz de usuario, muestra los datos y recibe las interacciones del usuario, delegando acciones al presentador.
- **Presentador** :Actúa como intermediario entre el modelo y la vista, manejando la lógica de presentación y actualizando la vista con datos del modelo.



The model-view-viewmodel (MVVM) pattern

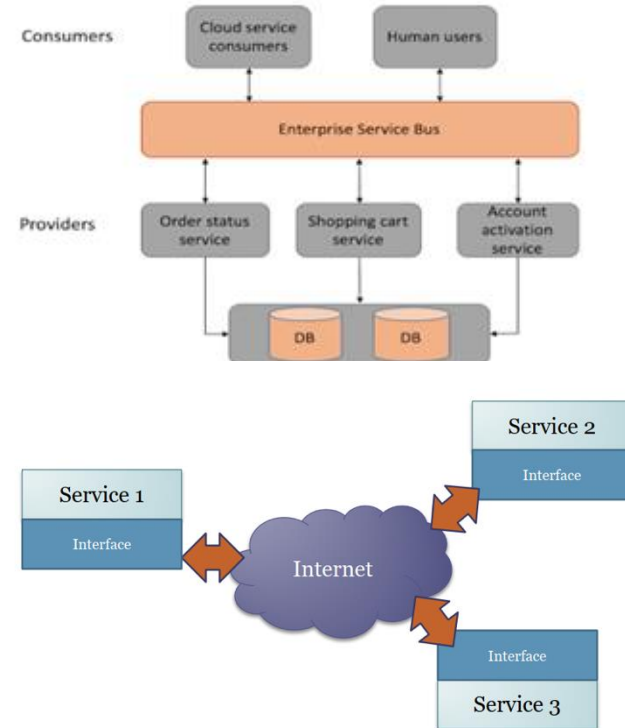
Separación entre los componentes Modelo y Vista

- **Modelo:** lógica de negocio y datos
- **Vista:** componentes de la interfaz de usuario como CSS, HTML. No realiza ninguna manipulación sobre los datos.
- **ViewModel:** mantiene la vista separada del modelo. Permite la interacción y coordinación entre la Vista y los componentes del modelo



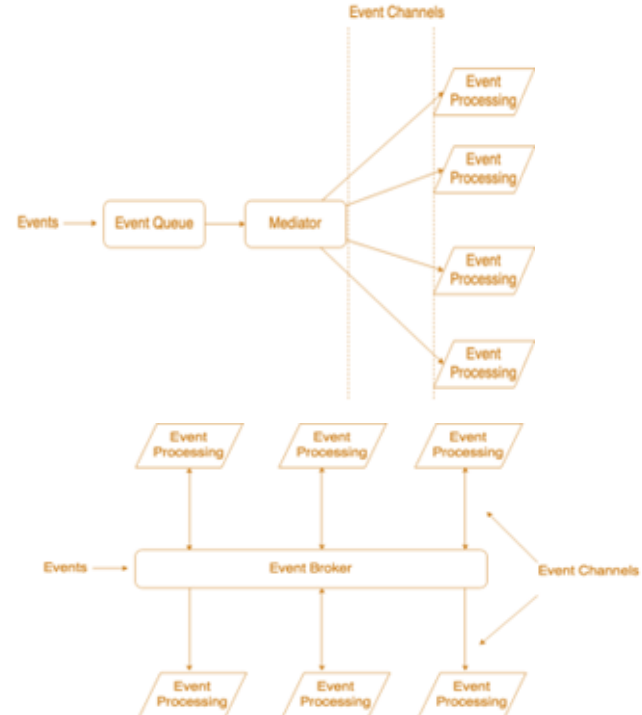
Service-oriented architecture (SOA)

La Arquitectura Orientada a Servicios (SOA) es una estrategia para construir sistemas de software centrados en el negocio a partir de bloques de construcción interoperables y poco acoplados (llamados Servicios) que se pueden combinar y reutilizar rápidamente, dentro y entre empresas, para satisfacer las necesidades comerciales.



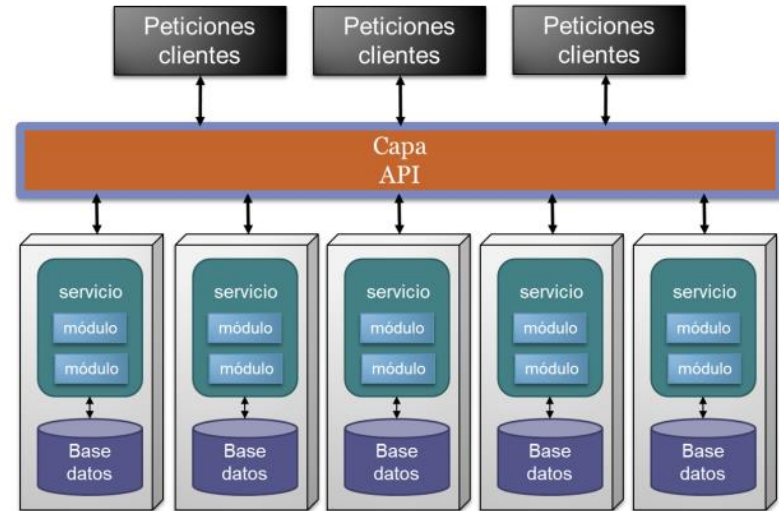
Event-driven architecture (EDA)

- Es una arquitectura asíncrona y distribuida, pensada para crear aplicaciones altamente escalables.
- Los componentes no se comunican de forma tradicional.
- En esta arquitectura, se espera que las aplicaciones lancen diversos “eventos” para que otros componentes puedan reaccionar a ellos.



Arquitectura Orientada a Microservicios

- Consiste en crear pequeños componentes de software que solo hacen una tarea, la hace bien y son totalmente autosuficientes
- Los Microservicios son Altamente Cohesivos
- Los Microservicios se comunican con otros Microservicios para delegar ciertas tareas.



Casos de Estudio



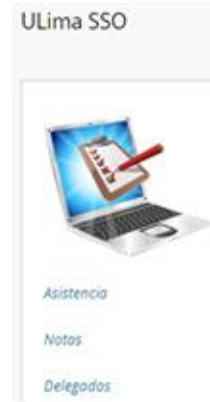
Caso 1 :

Analiza la aplicación Blackboard, que se utiliza en el curso de Ingeniería de Software, y determina cuáles son los atributos de calidad más relevantes para su funcionamiento.

Una vez identificados estos atributos, elige uno de ellos y elabora un escenario de calidad que ilustre su importancia. Por ejemplo, si seleccionas el rendimiento, describe un escenario en el que un gran número de estudiantes accede simultáneamente a la plataforma para participar en un examen en línea. Detalla cómo un rendimiento óptimo, que permita tiempos de carga rápidos y una respuesta ágil del sistema, puede garantizar una experiencia fluida y sin interrupciones, lo que es crucial para el éxito del examen y la satisfacción del usuario.

Requisitos Funcionales significativos para la arquitectura

- Un requerimiento funcional significativo para la arquitectura tiene que tener dos características esenciales:
 - Valor para el negocio.
 - Impacto en los atributos de calidad.
- En inglés
 - Architecturally Significant Requirements (ASR)



¿Cuáles son los Requisitos Funcionales significativos para la arquitectura?

—
¿Quiénes son los interesados claves?



¿Profesor?

ULima SSO



Asistencia

Notas

Delegados



¿Alumnos?

¿Cuáles son los Requisitos Funcionales significativos para la arquitectura?

¿Cuáles son los requisitos más usados por el profesor?



¿Profesor?



¿Cuáles son los Requisitos Funcionales significativos para la arquitectura?

¿Cuáles son los atributos de calidad más valorados por el profesor en relación a los requisitos más usados por él?

Seguridad

Que una vez confirmadas las notas, nadie pueda cambiar las notas y la asistencia

Rendimiento

Que no demore la subida o la descarga de un documento

Confiabilidad

Dado que ingresar notas es una tarea tediosa, quisiera que no ocurran fallas al grabar

Usabilidad

Que sea fáciles de usar para que no se cierre la sesión mientras ingreso las notas y la asistencia



¿Cuáles son los Requisitos Funcionales significativos para la arquitectura?

¿Cuáles son los atributos de calidad más valorados por el profesor en relación a los requisitos más usados por él?



¿Cuáles son los Requisitos Funcionales significativos para la arquitectura?

	Seguridad	Usabilidad	Rendimiento	Confiabilidad	Interoperabilidad
Registro de notas parciales	x	x		x	
Registro de notas finales	x	x		x	
Registro de asistencia		x		x	
Subir documentos			x		x
Enviar correos	x		x		x

Podemos escoger
entre Registro de
notas parciales y de
notas finales

Requisitos NO Funcionales significativos para la arquitectura

Escenario de calidad	#1
Atributo de calidad	Desempeño
Fuente del estímulo	Cuando 1000 profesores de forma concurrente
Estímulo	Suben un archivo de 15 MB
Entorno	Con una velocidad de subida en internet entre 1 y 1.5Mbps
Artefacto	A través de la página para subir archivos del sistema del aula virtual
Respuesta	El archivo es subido y puesto a disposición de lo estudiantes
Medida de la respuesta	95 % de los archivos están disponibles en menos de 5 segundos.

Todos los atributos de calidad son significativos par la arquitectura.



Caso 2 :

Una empresa de comercio electrónico, llamada **E-Shop**, ha experimentado un crecimiento significativo en su base de clientes y en el volumen de pedidos. Actualmente, utiliza una arquitectura monolítica para su sistema de gestión de pedidos, lo que ha llevado a problemas de rendimiento y dificultades en el mantenimiento. La empresa enfrenta desafíos como tiempos de carga lentos, dificultades para implementar nuevas características y una alta tasa de fallos en el sistema durante picos de tráfico.

Desafíos Identificados:

- **Dificultad para Escalar:** La arquitectura monolítica no permite escalar componentes individuales, lo que resulta en un rendimiento deficiente durante períodos de alta demanda, como durante las ventas especiales.
- **Lentitud en el Desarrollo:** Los equipos de desarrollo deben trabajar en el mismo código base, lo que provoca cuellos de botella y retrasa la implementación de nuevas funcionalidades.
- **Impacto de Fallos:** Un fallo en un componente del sistema puede afectar a toda la aplicación, lo que resulta en una mala experiencia para el usuario.
- **Limitaciones Tecnológicas:** La empresa se ve limitada a un solo stack tecnológico, lo que dificulta la adopción de nuevas tecnologías que podrían mejorar el rendimiento y la eficiencia.



Solución :

Para abordar estos desafíos, E-Shop debe adoptar una ***arquitectura de microservicios***. Esta nueva arquitectura se basa en la creación de servicios independientes que se comunican entre sí a través de APIs. Los microservicios propuestos incluyen:

- **Servicio de Gestión de Pedidos:** Encargado de crear, actualizar y rastrear pedidos.
- **Servicio de Gestión de Usuarios:** Responsable de la administración de cuentas de clientes.
- **Servicio de Procesamiento de Pagos:** Maneja las transacciones financieras.
- **Servicio de Notificaciones:** Envía actualizaciones a los clientes sobre el estado de sus pedidos.

Solución :

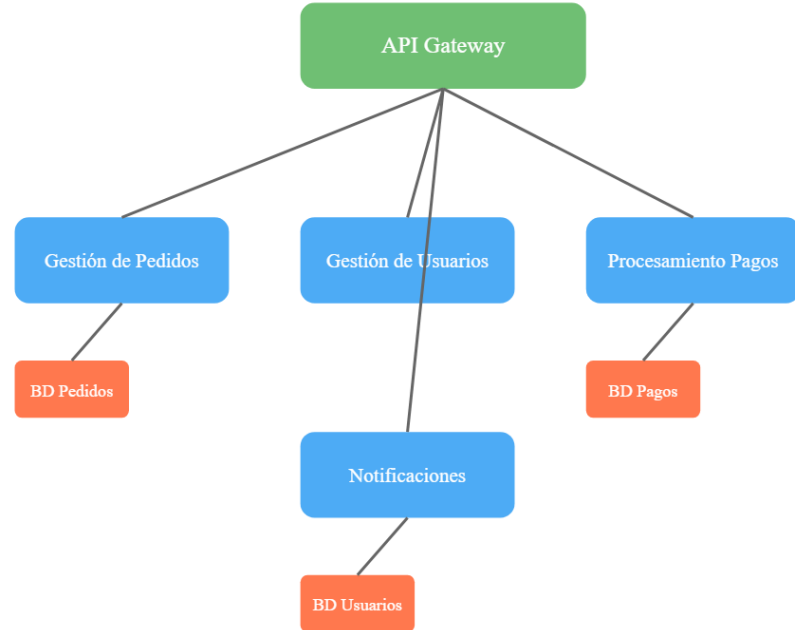
Beneficios del estilo seleccionado:

- **Escalabilidad:** Cada microservicio puede escalar de manera independiente, permitiendo a E-Shop manejar un mayor volumen de pedidos sin afectar el rendimiento general.
- **Desarrollo Ágil:** Los equipos pueden trabajar en diferentes microservicios simultáneamente, lo que acelera el ciclo de desarrollo y permite implementar nuevas características más rápidamente.
- **Resiliencia:** Si un microservicio falla, los demás pueden seguir funcionando, lo que minimiza el impacto en la experiencia del usuario.
- **Flexibilidad Tecnológica:** E-Shop puede utilizar diferentes tecnologías para cada microservicio, eligiendo las herramientas más adecuadas para cada función.



Solución :

1. El API Gateway actúa como punto de entrada único para todas las solicitudes de los clientes
2. Los cuatro microservicios principales están representados en azul
3. Cada microservicio tiene su propia base de datos (en naranja)
4. Las líneas representan la comunicación entre los componentes
5. La arquitectura está diseñada para ser escalable y mantener la independencia entre servicios



Caso 3 :

Un hospital grande, llamado **HealthCarePlus**, está buscando modernizar su sistema de gestión. Actualmente, utiliza un sistema heredado que centraliza toda la información en un único servidor. Este sistema maneja funciones como la gestión de pacientes, programación de citas, administración de medicamentos, facturación y reportes médicos. Sin embargo, el sistema presenta varios problemas:

- **Baja disponibilidad:** Si el servidor central falla, todo el sistema queda inoperativo, lo que afecta la atención a los pacientes.
- **Escalabilidad limitada:** Durante horas pico, como en emergencias o temporadas de alta demanda, el sistema se vuelve lento y no puede manejar el volumen de solicitudes.
- **Dificultad para integrar nuevos módulos:** Cada vez que se necesita agregar una nueva funcionalidad, como un sistema de telemedicina, el desarrollo y la integración son lentos y costosos.
- **Falta de flexibilidad:** Los diferentes departamentos del hospital (emergencias, farmacia, administración, etc.) tienen necesidades específicas, pero el sistema actual no permite personalización ni independencia entre módulos.

El hospital necesita un sistema que sea **escalable, resiliente**, y que permita a los diferentes departamentos trabajar de manera independiente, pero manteniendo la comunicación entre ellos. Determina cuál sería el estilo arquitectónico más adecuado para resolver los problemas actuales. Justifica tu elección.



Solución :

Beneficios de la Solución :

- 1. Independencia de Componentes:** Cada módulo puede ser desarrollado, desplegado y mantenido de forma independiente, lo que reduce la complejidad del sistema.
- 2. Flexibilidad:** Los departamentos pueden trabajar de manera autónoma, pero los componentes siguen comunicándose para compartir información relevante.
- 3. Escalabilidad:** Los recursos pueden asignarse dinámicamente a los componentes que más lo necesiten, optimizando el rendimiento.
- 4. Resiliencia:** El sistema es más robusto frente a fallos, ya que un problema en un componente no afecta a los demás.
- 5. Evolución del Sistema:** Es más fácil agregar nuevas funcionalidades o reemplazar componentes obsoletos sin interrumpir el funcionamiento general.

Solución :

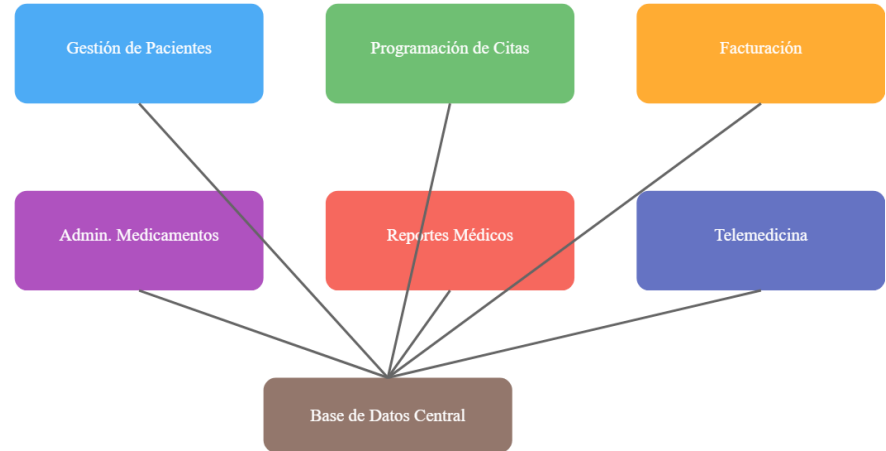
El estilo arquitectónico más adecuado para este caso es la **Arquitectura Basada en Componentes**. Este enfoque divide el sistema en módulos independientes (componentes) que se comunican entre sí a través de interfaces bien definidas. Cada componente es responsable de una funcionalidad específica, como la gestión de pacientes, la programación de citas o la facturación.

La elección de este estilo se justifica por:

- **Modularidad:** Cada departamento del hospital puede tener su propio componente, lo que permite personalizar las funcionalidades según sus necesidades específicas.
- **Escalabilidad:** Los componentes pueden ser desplegados y escalados de manera independiente. Por ejemplo, el componente de emergencias puede recibir más recursos durante picos de demanda.
- **Resiliencia:** Si un componente falla (por ejemplo, el de facturación), los demás componentes pueden seguir funcionando, garantizando la continuidad del servicio.
- **Facilidad de Integración:** Nuevos módulos, como un sistema de telemedicina, pueden ser desarrollados e integrados como nuevos componentes sin afectar el resto del sistema.
- **Mantenibilidad:** Al estar separados, los componentes son más fáciles de actualizar, probar y mantener.

Solución :

- **Componentes Independientes:** Cada rectángulo representa un componente que maneja una funcionalidad específica del sistema
- **Base de Datos Central:** Todos los componentes comparten una base de datos central para garantizar la consistencia de la información
- **Conexiones:** Las líneas representan la comunicación entre los componentes y la base de datos central.



¿Qué se logró?



- ✓ Comprenden la arquitectura de software y su impacto en la calidad del sistema.
- ✓ Entender las etapas del ciclo de desarrollo de la arquitectura de software.
- ✓ Identifican drivers arquitectónicos y crean escenarios de calidad para justificar decisiones de diseño.
- ✓ identificar y analizan diferentes estilos arquitectónicos, comprendiendo sus características, ventajas y desventajas, lo que les permite seleccionar el más adecuado para diversos contextos de desarrollo

¿Qué debemos hacer para alcanzar el objetivo del tema?



- ✓ Repasar la teoría, los ejemplos y casos resueltos.



Referencias Bibliográficas

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design Patterns: Elements of Reusable Object-Oriented Software Addison-Wesley.
- <https://github.com/RameshMF/gof-java-design-patterns>
- <https://refactoring.guru/design-patterns/abstract-factory/java/example>
- <https://refactoring.guru/es/design-patterns/abstract-factory/java/example>
- <http://www.cs.sjsu.edu/faculty/pearce/modules/patterns/creational/factory.htm>
- <https://www.oscarblancarteblog.com/2014/07/18/patron-de-diseno-factory/>

GRACIAS